

When Does Depth Recursion Pay? A Parameter-Matched Study of Weight-Tied Hybrid Transformers on 100M Unique Words

Aditya Sasidhar

Vellore Institute of Technology
telikicherlaadityasasidhar@gmail.com

Abstract

Weight-tied depth recursion promises effective depth without parameters, but controlled evidence at small scale is thin: recursion is usually evaluated by *compressing* a pre-trained model, not by pre-training a recursive model against an exactly parameter-matched, non-recursive twin. We pre-train five hybrid Gated DeltaNet (GDN) + grouped-query attention (GQA) transformers from scratch on the BabyLM 2026 Strict corpus (100M words) under one locked recipe — identical tokenizer, data order, optimizer, and 500M-token budget — including **two exact parameter-matched ablation pairs** in which weight-tied recursion (two super-blocks, each applied $R = 3$ times) is the *only* difference, plus a larger pure-GQA baseline. Recursion wins both controlled ablations: the recursive members beat their twins by 0.026 and 0.019 nats of validation loss and by 3.31 and 4.21 points on zero-shot BLiMP, while reaching their twins’ final loss with 21% and 16% fewer training tokens. The benefit is not immediate: both recursive models trail their twins through the first pass over the corpus and take the lead only once data repeats. Rankings across architectures are metric-dependent, however: the recursive models rank first and second on validation loss, whereas the pure-GQA baseline leads BLiMP at 71.07. Position-wise loss analysis locates recursion’s advantage at long range, but the BLiMP reversal cautions that next-token loss alone does not establish attention density as unimportant. We also document an evaluation-weighting pitfall that initially masked part of these results. Code, logs, checkpoints, and evaluation outputs are released; training the full grid costs about 3 H100-hours.

1 Introduction

The BabyLM challenge fixes the training data — roughly the number of words a human encounters during development — and asks what modeling

choices make the most of it (BabyLM Organizing Committee, 2026). Most entries respond with data curricula or training objectives. We instead use the fixed-data setting for what it does best: making an *architectural* question unusually clean to answer.

If parameters are scarce, is it better to spend them on independent layers, or to re-apply a smaller stack of layers several times with tied weights?

Weight-tied depth recursion — applying the same block of layers repeatedly — decouples a model’s *effective depth* from its parameter count. The idea is old (Dehghani et al., 2019; Lan et al., 2020) and newly relevant for parameter-efficient LLMs (Bae et al., 2024, 2025). But recent work mostly *converts* pre-trained models into recursive ones. What is missing is the direct experiment: pre-train a recursive model from scratch against a baseline that matches its parameter count exactly and differs in nothing else. Small-data pre-training is precisely where the answer matters most — and where the experiment is cheap enough to run properly.

We run it inside a modern *hybrid* architecture that interleaves Gated DeltaNet linear-attention layers (Yang et al., 2025) with softmax grouped-query attention layers (Ainslie et al., 2023) in fixed ratios, following the design popularized by Qwen3-Next (Qwen Team, 2025). Hybrids are a natural host for the question: a hybrid *ratio unit* (e.g., GDN–GDN–GQA) is a self-contained token-mixing block, so recursion has an obvious granularity — repeat the unit. They also let us test a second, related hypothesis: that small models are constrained by the *density of softmax attention* — the intuition that precise, unbounded-memory retrieval is what a small budget can least afford to cut.

Contributions.

1. **A controlled recursion ablation.** Two *exact* parameter-matched pairs (46,913,888 and 63,487,504 non-embedding parameters; zero-parameter difference within each pair) in which the recursive member re-applies each of two super-blocks three times with tied weights and the non-recursive member applies the same layers once — with tokenizer, data, data *order*, optimizer, schedule, and token budget identical across all runs by construction.
2. **Recursion wins both ablations — in loss and BLiMP.** The recursive members beat their twins by 0.026 / 0.019 nats and 3.31 / 4.21 BLiMP points, reach their twins’ final loss with 21% / 16% fewer tokens, and finish first and second on validation loss. Both trail their twins at the 100M-token checkpoint and overtake only during the second pass over the corpus: the loss advantage of tied depth compounds with exposure.
3. **Depth helps consistently within pairs, but model rankings depend on the metric.** Position-wise loss analysis shows the recursion gap widening with context position and no long-context advantage from the all-attention baseline. Yet that baseline leads BLiMP at 71.07, ahead of Recursive 3:1 at 67.48. Thus iterated depth is a robust parameter-efficiency lever inside the controlled hybrid pairs, while next-token loss and grammatical minimal-pair accuracy reward different architectural properties.
4. **A documented evaluation pitfall.** Our training-time validation logger averaged per-*batch* means with batch size tied to a memory setting, silently biasing reported losses by -0.025 for some runs and initially flipping one ablation’s sign. All numbers here are recomputed from checkpoints under uniform per-token weighting; the pitfall and fix are documented for other small-budget practitioners (Appendix A).

2 Related Work

Parameter sharing across depth. Universal Transformers (Dehghani et al., 2019) apply one block recurrently with adaptive halting; ALBERT (Lan et al., 2020) ties all encoder layers, trading

a little quality for large parameter savings. Recent work revisits tying for decoder-only LLMs: Relaxed Recursive Transformers (Bae et al., 2024) initialize a recursive model from a pre-trained one and relax the tie with layer-wise LoRA; Mixture-of-Recurions (Bae et al., 2025) learns token-level recursion depths. These lines evaluate recursion as a *conversion* or *routing* mechanism on top of models trained conventionally; we isolate recursion itself, from scratch, at exactly matched parameters.

Linear attention and the delta rule. DeltaNet-style linear attention maintains a fast-weight state updated by the delta rule and is parallelizable over sequence length (Yang et al., 2024). Gated DeltaNet adds a data-dependent decay gate and improves over Mamba2 and DeltaNet (Yang et al., 2025). We use the reference Triton kernels from the flash-linear-attention library (Yang and Zhang, 2024).

Hybrid architectures. A minority of softmax-attention layers interleaved with a majority of linear-attention layers preserves most of full attention’s quality at a fraction of its inference cost; Qwen3-Next (Qwen Team, 2025) adopts a 3:1 Gated-DeltaNet:attention ratio at 80B scale. We evaluate 2:1 and 3:1 ratios at BabyLM scale and use the ratio unit as the unit of recursion.

Sample-efficient pre-training. The BabyLM workshop series (Warstadt et al., 2023; BabyLM Organizing Committee, 2026) fixes the corpus (Strict track: 100M words; repetition allowed up to 1B words of total exposure) so that architectural and algorithmic effects are comparable across submissions. Our runs consume ~ 295 M words of exposure each, well within budget — and, as it turns out, the repetition allowance is load-bearing for our result (Section 6).

3 Models

All five variants share one spine: a decoder-only pre-norm transformer with $d_{\text{model}} = 768$, RMSNorm (Zhang and Sennrich, 2019, $\epsilon = 10^{-5}$), SwiGLU feed-forward blocks (Shazeer, 2020) with $d_{\text{ff}} = 2304$ (a fixed $3\times$ multiplier for *all* variants), tied input/output embeddings, and the BabyLM-community BPE tokenizer (vocabulary 16,384). Every token-mixing layer is followed by one SwiGLU FFN sub-layer with its own residual connection.

GQA layers use 12 query heads and 4 KV heads of dimension 64, rotary position embeddings (Su et al., 2021) with base 10,000, and per-head RMS QK-normalization (Henry et al., 2020). Softmax attention runs FlashAttention-3 on Hopper GPUs (Shah et al., 2024) with an SDPA fallback elsewhere.

GDN layers are Gated DeltaNet layers (Yang et al., 2025) as implemented in flash-linear-attention v0.5.1: 12 heads of dimension 64, value expansion 1, output gating and short (size-4) depthwise convolutions enabled, and a delta-rule fast-weight state with data-dependent decay. GDN applies its own L2 normalization to queries and keys.

3.1 The five variants

A *ratio unit* is (GDN, GDN, GQA) for 2:1 and (GDN, GDN, GDN, GQA) for 3:1. The non-recursive hybrids stack two units. The recursive hybrids contain **two independent super-blocks in sequence, each holding one ratio unit, each applied $R = 3$ times with weights tied within that super-block** (never across super-blocks). Unrolled, Recursive 2:1 computes 18 layer applications with the parameters of 6 layers — and, notably, applies softmax attention 6 times per token where its twin applies it twice.

This yields the design property the study rests on: within each pair, the two models have identical layer inventories and identical parameter counts *to the digit*, and differ only in whether the super-blocks are applied once or three times (Table 1). The pure-GQA baseline is a no-GDN reference at its own, larger size; we use it as a size anchor and as the maximum-attention-density point in the analysis of Section 5.2.

3.2 Initialization

Weights are drawn from $\mathcal{N}(0, 0.02^2)$; residual-output projections (attention output, FFN down-projection) are scaled by $1/\sqrt{2D_{\text{eff}}}$, where D_{eff} is the *effective* depth — 18 or 24 for the recursive models, the plain layer count otherwise. Using effective rather than unique depth matters under tying: the same projection writes into the residual stream R times.

4 Experimental Setup

Data. We train on the official BabyLM 2026 Strict corpus (six domain files, $\sim 100\text{M}$ words) and

validate on the official dev split, excluded from the training bins at preprocessing time.¹ Under the shared 16,384-vocabulary tokenizer, one epoch is 169,741,563 tokens (≈ 1.70 tokens per whitespace word).

Recipe (identical for all five runs). Sequence length 4,096; batch 32 sequences = 131,072 tokens per optimizer step; 3,814 steps = 499,908,608 tokens ≈ 2.9 epochs $\approx 295\text{M}$ words of exposure — compliant with the Strict-track rules ($\leq 100\text{M}$ unique words; $\leq 1\text{B}$ words of total exposure). AdamW (Loshchilov and Hutter, 2019) with $\beta = (0.9, 0.95)$, weight decay 0.1 on matrices only, gradient clipping at 1.0; peak learning rate 6×10^{-4} with 250 warmup steps and cosine decay to 10% of peak; bf16 autocast with fp32 master weights; seed 42. Data order is identical across variants by construction: every run draws the same seeded shuffle of 4,096-token chunks each epoch, so all five models see exactly the same tokens in exactly the same order.

Evaluation protocol. All validation numbers reported in this paper are recomputed from the retained checkpoints (one per 100M training tokens, plus final) over the same fixed 2M-token slice of the dev split: 488 non-overlapping 4,096-token windows, uniform per-token weighting, identical code path for every model. We do *not* use the training-time logger’s values: it averaged per-batch means with batch size tied to a per-run memory setting, which biased four runs’ logged losses by -0.025 relative to the fifth — enough to invert one ablation’s apparent sign. Appendix A documents the pitfall.

Hardware and cost. Each variant trains on a single H100-80GB (Modal): 18.8–27.6 min for the non-recursive models, 46.2 and 62.1 min for Recursive 2:1 and 3:1 — about 3 GPU-hours for the grid. Gradient accumulation splits the 32-sequence batch into micro-batches of 16 (8 for Recursive 3:1, whose 24-layer effective depth needs a smaller activation footprint); this leaves the optimizer-step mathematics unchanged.

5 Results

Table 2 reports final validation metrics after the full 500M-token budget; Table 3 and Figure 1

¹BabyLM-community/BabyLM-2026-Strict and BabyLM-community/BabyLM-dev on the HuggingFace Hub.

Variant	Attention mix	Unique layers	Super-blocks $\times$ R	Eff. depth	Non-emb. params	Total params
GQA baseline	pure GQA	10	—	10	68,830,208	81,413,120
GDN 2:1	GDN:GQA = 2:1	6	—	6	46,913,888	59,496,800
GDN 3:1	GDN:GQA = 3:1	8	—	8	63,487,504	76,070,416
Recursive 2:1	GDN:GQA = 2:1	6	2×3	18	46,913,888	59,496,800
Recursive 3:1	GDN:GQA = 3:1	8	2×3	24	63,487,504	76,070,416

Table 1: The five pre-trained variants. GDN 2:1 / Recursive 2:1 and GDN 3:1 / Recursive 3:1 are exact parameter-matched pairs.

trace the trajectories; Figure 2 summarizes the parameter–quality trade-off; Table 5 breaks the losses down by context position; and Table 4 reports zero-shot BLiMP.

Recursion wins both ablations. Against identically parameterized, identically trained twins, recursion improves final validation loss by 0.026 nats (2.6% perplexity) in the 2:1 pair and 0.019 nats (2.0%) in the 3:1 pair. The two recursive models finish first and second among all five; the pure-GQA baseline beats only the shallow GDN 2:1. The 46.9M Recursive 2:1 outperforms the 68.8M baseline (0.017 nats) with 32% fewer non-embedding parameters, and the 63.5M Recursive 3:1 beats it by 0.032 nats (3.2% perplexity). The gap is larger where the twin is shallower (6 layers: 0.026; 8 layers: 0.019), consistent with recursion buying the most where the parameter budget forces the least depth.

The advantage needs repetition. The recursive models are slow starters: at the 100M-token checkpoint — inside the first pass over the corpus — each trails its own twin (by 0.009 and 0.034 nats; Table 3). By 200M tokens, early in the second epoch, both lead, and the pair gaps then widen monotonically through the end of training. Interpolating on the checkpoint grid, Recursive 2:1 reaches its twin’s final loss with $\sim 21\%$ fewer training tokens ($\approx 396\text{M}$ vs. 500M) and Recursive 3:1 with $\sim 16\%$ fewer ($\approx 421\text{M}$); Recursive 3:1 reaches the *baseline*’s final loss with $\sim 23\%$ fewer. An evaluation at the single-epoch point — the unique-data limit — would have concluded that recursion strictly hurts.

Fitting and generalization now agree. The recursive member also has the lower *training* loss in both pairs (2.588 vs. 2.614; 2.552 vs. 2.568): the extra effective depth adds fitting capacity, and under this budget that capacity transfers to held-out data rather than overfitting the (repeated) corpus.

Stability and cost. All five runs completed with zero loss spikes and gradient norms settling at 0.24–0.26. The recursive models pay for their quality in compute, not parameters: at fixed parameters they execute $\sim 3\times$ the layer FLOPs per token, and throughput drops accordingly (181k and 134k tokens/s vs. 443k and 353k for their twins; Appendix A).

5.1 Zero-shot grammatical generalization

We evaluate each final checkpoint on the official BabyLM 2026 full BLiMP and BLiMP-supplement sets (Warstadt et al., 2020): 59,875 and 5,218 minimal pairs, respectively. Following the official causal protocol, candidates are ranked by summed log-probability over all non-BOS sentence tokens, and scores are macro-averaged over the 67 BLiMP paradigms (five supplement tasks). Table 4 reports accuracy at temperature 1.

The controlled result generalizes beyond LM loss. Recursion adds 3.31 and 4.21 BLiMP points within the 2:1 and 3:1 pairs, with smaller but consistently positive supplement gains (1.01 and 1.24). The larger BLiMP gain in the 3:1 pair contrasts with its smaller validation-loss gain, so recursion’s effect is not merely a monotone rescaling of held-out loss.

The across-architecture ranking reverses. The pure-GQA baseline, fourth on validation loss, ranks first on both grammatical suites (71.07 / 59.59). Recursive 3:1 closes much of the gap and remains the strongest hybrid, but does not overtake the larger all-attention model. Because the baseline differs in both parameter count and attention composition, this does not isolate which factor causes the reversal; it does show that dev loss alone is an incomplete model-selection criterion.

Model	Non-emb. params	Eff. depth	Val loss ↓	Val ppl ↓	bpt ↓	Train loss
Recursive 3:1	63.5M	24	3.0926	22.03	4.462	2.552
Recursive 2:1	46.9M	18	3.1072	22.36	4.483	2.588
GDN 3:1	63.5M	8	3.1121	22.47	4.490	2.568
GQA baseline	68.8M	10	3.1244	22.75	4.508	2.609
GDN 2:1	46.9M	6	3.1333	22.95	4.520	2.614

Table 2: Final metrics on the fixed dev slice after 500M training tokens (uniform per-token weighting; single seed). Loss in nats/token; bpt = bits per token. Rows sharing a parameter count are exact recursion ablations.

Model	100M	200M	300M	400M	500M
Recursive 2:1	3.5121	3.2778	3.1803	3.1315	3.1072
GDN 2:1	3.5033	3.2923	3.2033	3.1564	3.1333
Recursive 3:1	3.5292	3.2712	3.1676	3.1174	3.0926
GDN 3:1	3.4952	3.2739	3.1803	3.1352	3.1121
GQA baseline	3.5560	3.3041	3.2052	3.1505	3.1244
<i>pair Δ (2:1, gdn – rec)</i>	−0.009	+0.014	+0.023	+0.025	+0.026
<i>pair Δ (3:1, gdn – rec)</i>	−0.034	+0.003	+0.013	+0.018	+0.019

Table 3: Validation loss (nats) at each retained checkpoint (uniform weighting; one epoch $\approx$ 170M tokens; bold = leader within pair at 100M, overall leader thereafter). Both recursive models trail their twins inside the first epoch, lead by 200M tokens, and the pair gaps widen monotonically to the end of the budget.

Model	BLiMP ↑	Supplement ↑
GQA baseline	71.07	59.59
Recursive 3:1	67.48	55.17
Recursive 2:1	65.56	53.30
GDN 3:1	63.27	53.93
GDN 2:1	62.24	52.28
Δ 2:1 (rec – gdn)	+3.31	+1.01
Δ 3:1 (rec – gdn)	+4.21	+1.24

Table 4: Zero-shot grammatical minimal-pair accuracy on the official BabyLM 2026 full evaluation sets. Scores macro-average paradigms/tasks. Within both exact parameter-matched pairs, recursion improves BLiMP and the supplement; bold marks the overall leader.

5.2 Is softmax-attention density what small models lack?

A plausible reading of hybrid designs is that softmax attention — with its unbounded, token-precise memory — is the scarcest resource in a small model, and that GDN layers (whose per-head state is a fixed 64×64 fast-weight matrix) are the compressible filler. Under that hypothesis, the all-attention baseline should shine, hybrids with only two softmax layers should struggle at long range, and recursion’s benefit should be read as merely restoring softmax applications ($2 \rightarrow 6$ per token). Our grid spans the extremes of this axis — 100% attention density (baseline, 10 softmax applications per token), 25–33% (hybrids, 2 applications), and recursive hybrids (6 applications).

The position-wise *validation-loss* data rejects density as the binding constraint for long-context next-token prediction, on three counts; the BLiMP result above limits that conclusion to this metric.

First, density does not predict quality. The 100%-attention baseline loses to the 25%-density GDN 3:1 at comparable size (3.1244 vs. 3.1121) and beats only the 6-layer GDN 2:1. The two hybrids have *identical* softmax budgets — two applications each — yet sit at opposite ends of the non-recursive ranking; what separates them is depth and parameters, not attention.

Second, density does not predict long-context utilization. If two softmax layers were a retrieval bottleneck, the hybrids should benefit least from added context. They do not: the baseline’s long-range gain (0.0202) is indistinguishable from the two-softmax hybrids’ (0.0200, 0.0204; Table 5). At a 4,096-token context and this scale, two softmax layers appear to be *enough* — additional attention density earns no measurable long-range return. (The baseline’s late-bucket loss even ticks up from 1K–2K to 2K–4K, the only model where it does.)

Third, what does improve long-range prediction is recursion. The recursive models post the largest long-range gains (0.0237, 0.0230), and the within-pair recursion advantage *widens* with position — from 0.0240 to 0.0272 nats across buckets

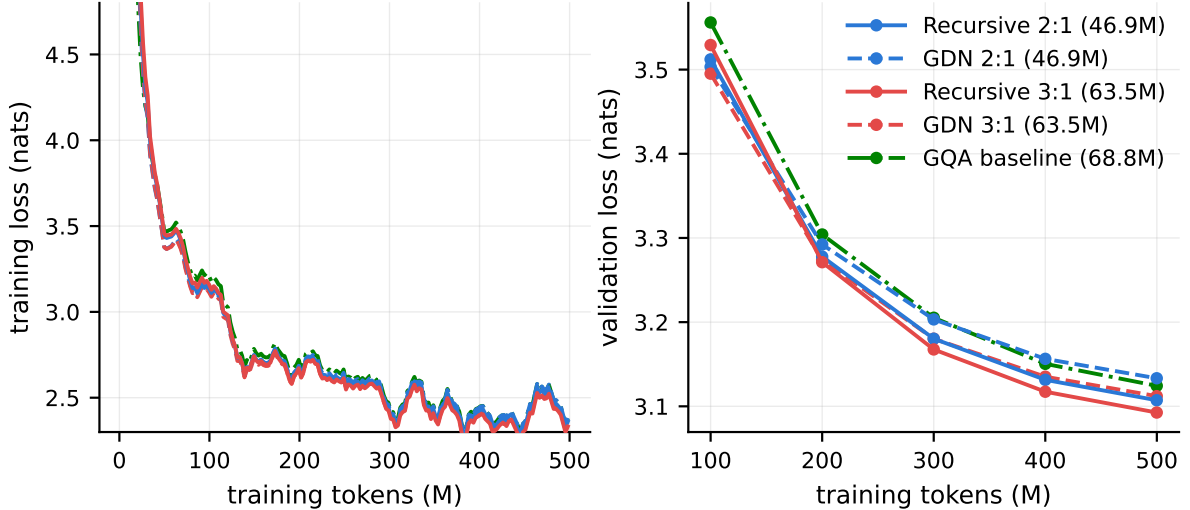


Figure 1: Training loss (left, smoothed, from training logs) and checkpoint validation loss (right, uniform protocol) versus training tokens. Hue encodes the ablation pair (blue = 2:1, red = 3:1, green = baseline); solid lines are recursive models, dashed are their non-recursive twins. All runs are stable — zero loss spikes across 19,070 optimizer steps in total — and every model is still improving when the budget ends.

Model	Softmax apps	0–256	256–1K	1K–2K	2K–4K	Long-range gain
Recursive 3:1	6	3.2516	3.0998	3.0793	3.0768	0.0230
Recursive 2:1	6	3.2640	3.1148	3.0944	3.0912	0.0237
GDN 3:1	2	3.2675	3.1172	3.0993	3.0972	0.0200
GQA baseline	10	3.2862	3.1299	3.1093	3.1096	0.0202
GDN 2:1	2	3.2884	3.1388	3.1204	3.1184	0.0204

Table 5: Final-checkpoint validation loss by context position (nats; same 488-window slice). “Softmax apps” = softmax-attention applications per token (effective). “Long-range gain” = $\text{loss}(256\text{--}1\text{K}) - \text{loss}(2\text{K--}4\text{K})$: how much a model improves when given long context.

in the 2:1 pair and from 0.0174 to 0.0204 in the 3:1 pair. Since recursion triples softmax applications *and* GDN state updates *and* FFN transformations alike, and since the attention-dense baseline shows no such long-range edge, the parsimonious reading is that *iterated processing of the same context* — multi-pass refinement, in the spirit of Universal Transformers — is what improves long-range prediction here, rather than attention count per se.

For small-budget practitioners the metric matters: two well-placed attention layers suffice for 4K-context next-token loss, and weight tying buys useful effective depth at fixed parameters; the BLiMP ranking nevertheless leaves room for dense softmax attention to benefit grammatical generalization.

6 Discussion

Recursion as a parameter-efficiency lever.

Both ablations favor recursion on validation loss and BLiMP. For loss, the larger payoff occurs

where the twin is shallower (0.026 nats at 6 unique layers vs. 0.019 at 8), and ordering models by effective depth nearly predicts the full loss ranking. For BLiMP, however, the 3:1 pair gains more (+4.21 vs. +3.31 points) and the 10-layer GQA baseline leads. The robust conclusion is therefore within-pair: *when parameters are fixed, tied computation improves both predictive fit and grammatical minimal-pair accuracy*; effective depth alone does not predict performance across architectures.

Metric-dependent architecture ranking. Validation loss rewards the recursive hybrids, while BLiMP rewards the larger all-attention baseline. BLiMP morphology shows the largest baseline margin (85.60 vs. 78.80 for Recursive 3:1), whereas the recursive models retain their controlled advantage across the aggregate. This divergence argues against selecting small models from dev loss alone and motivates reporting both distributional fit and targeted linguistic evaluation.

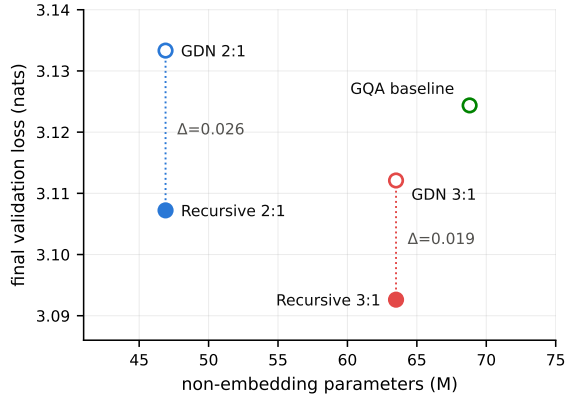


Figure 2: The parameter–quality trade-off. Recursion improves both parameter-matched pairs ($\Delta = 0.026$ and 0.019 nats), and both recursive models beat every non-recursive model, including the larger pure-attention baseline.

Repetition is load-bearing. The BabyLM Strict track allows up to ten passes over the 100M-word corpus, and our trajectory data shows the recursion benefit is a *multi-epoch* phenomenon: inside the first pass, tied depth is strictly worse than untied depth at equal parameters; the sign flips early in the second pass and the gap then grows monotonically (Table 3). The extra optimization difficulty of a deep tied network is paid up front; the capacity it buys is realized only with continued exposure. This suggests our 2.9-epoch budget *understates* recursion’s advantage — every model was still improving at cutoff, and the recursive models were improving from in front. It also cautions against evaluating recursive architectures in single-epoch regimes.

Parameter-matched is not compute-matched.

Within each pair the recursive model uses $\sim 3\times$ the forward/backward FLOPs per token (unrolled footprint 140.7M vs. 46.9M; 190.5M vs. 63.5M). Our claim is therefore about parameter efficiency — the right frame when memory, deployment size, or per-parameter data efficiency is the constraint — not compute-optimal training. Compute alone does not explain the validation-loss ranking, however: the baseline (68.8M unrolled) and GDN 3:1 (63.5M) execute fewer FLOPs than Recursive 2:1 (140.7M) and lose to it, but GDN 3:1 also *beats* the more-FLOPs baseline, and Section 5.2 shows the recursive models’ loss edge has a specific, long-range signature rather than a uniform more-compute-better one.

Why is the 3:1 gap smaller than the 2:1 gap?

Two non-exclusive candidates. (i) *Depth adequacy*: at 8 unique layers the twin is less depth-starved, so borrowed depth buys less. (ii) *Optimization cost*: the 24-deep tied model pays the largest first-epoch penalty (-0.034 nats at 100M tokens, vs. -0.009 for the 18-deep one) and has the least budget left to amortize it; under a longer budget the 3:1 gap may still be growing (it rose from $+0.003$ at 200M to $+0.019$ at 500M with no sign of flattening).

7 Conclusion

In a tightly controlled from-scratch comparison on the BabyLM 2026 Strict corpus — two exact parameter-matched recursion ablations plus a larger pure-attention baseline, all under one locked recipe — weight-tied depth recursion won both controlled comparisons: 0.026 and 0.019 nats of validation loss and 3.31 and 4.21 BLiMP points over identically parameterized twins. The loss benefit emerges only with repeated data, and position-wise analysis locates it disproportionately at long range. Across architectures, however, the metrics disagree: recursive hybrids rank first and second on loss, while the larger pure-GQA baseline leads BLiMP at 71.07 . Weight tying is therefore a reliable way to trade compute for quality at fixed hybrid-model parameters, but the best architecture depends on whether the target is next-token fit or grammatical generalization. At about 3 GPU-hours for the training grid, the experiment is cheap to replicate and extend; we release code, logs, checkpoints, and evaluation outputs to make that easy.

Limitations

- **Single seed, small deltas.** Every number comes from one run per variant (fixed seed 42, identical data order). The pairwise gaps (0.019 – 0.026 nats) are small, though consistent in sign across the last four checkpoints and across context positions. Multi-seed replication is the highest-priority follow-up, and at ≤ 1 H100-hour per run it is practical.
- **Partial downstream suite.** We now report the full official BLiMP and BLiMP-supplement evaluations, but have not yet run the remaining BabyLM tasks (EWoK, COMPS, entity tracking, reading-time pre-

diction, AoA, and fine-tuned GLUE). Those tasks may produce further ranking reversals.

- **Parameter-matched, not compute-matched.** Recursive models use $\sim 3\times$ the training and inference FLOPs of their twins at equal parameters. A compute-matched comparison (e.g., twins trained $3\times$ longer) is out of scope here.
- **Coarse density axis.** The attention-density analysis (Section 5.2) contrasts only two softmax budgets (2 and 10 applications) at one context length (4,096); density could matter at longer contexts or on retrieval-heavy tasks that LM loss under-weights.
- **Narrow architectural slice.** One width (768), two sizes, two hybrid ratios, $R = 3$ only, and a 500M-token budget ($\sim 30\%$ of the exposure the Strict track allows). The learning rate (6×10^{-4}) was a literature prior shared by all runs rather than tuned per variant. Token-efficiency figures are interpolated on a 100M-token checkpoint grid.
- **Minor provenance asymmetries.** The baseline ran on PyTorch 2.12.1 vs. 2.13.0 for the other four (a dependency-pinning slip corrected mid-study), and Recursive 3:1 used gradient-accumulation micro-batches of 8 vs. 16 elsewhere. Both leave the optimizer-step mathematics and data order unchanged, and the reported evaluation is checkpoint-based and identical across models.

Acknowledgments

Training used the flash-linear-attention library and FlashAttention-3 kernels; compute was rented on Modal H100 instances.

References

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. ArXiv:2305.13245.
- BabyLM Organizing Committee. 2026. BabyLM turns 4: Call for papers for the 2026 BabyLM workshop. ArXiv:2602.20092.
- Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. 2024. Relaxed recursive transformers: Effective parameter sharing with layer-wise LoRA. ArXiv:2410.20672.
- Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. 2025. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. ArXiv:2507.10524.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. 2019. Universal transformers. In *International Conference on Learning Representations*. ArXiv:1807.03819.
- Alex Henry, Prudhvi Raj Dachapally, Shubham Pawar, and Yuxuan Chen. 2020. Query-key normalization for transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. ArXiv:2010.04245.
- Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2020. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations*. ArXiv:1909.11942.
- Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations*. ArXiv:1711.05101.
- Qwen Team. 2025. [Qwen3-next-80b-a3b: Towards ultimate training and inference efficiency](#). Model release and blog post, Alibaba Cloud.
- Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. ArXiv:2407.08608.
- Noam Shazeer. 2020. GLU variants improve transformer. ArXiv:2002.05202.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced transformer with rotary position embedding. ArXiv:2104.09864.
- Alex Warstadt, Aaron Mueller, Leshem Choshen, Ethan Wilcox, Chengxu Zhuang, et al. 2023. Findings of the BabyLM challenge: Sample-efficient pre-training on developmentally plausible corpora. In *Proceedings of the BabyLM Challenge at the 27th Conference on Computational Natural Language Learning*.
- Alex Warstadt, Alicia Parrish, Haokun Liu, Anhad Mo-hananey, Wei Peng, Sheng-Fu Wang, and Samuel R. Bowman. 2020. BLiMP: The benchmark of linguistic minimal pairs for English. *Transactions of the Association for Computational Linguistics*, 8:377–392.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2025. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*. ArXiv:2412.06464.

Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. 2024. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems*. ArXiv:2406.06484.

Songlin Yang and Yu Zhang. 2024. FLA: A triton-based library for hardware-efficient implementations of linear attention mechanisms. <https://github.com/fla-org/flash-linear-attention>.

Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*. ArXiv:1910.07467.

A Reproducibility details

Per-run provenance. All runs are in the Weights & Biases project `babylm-2026`; Table 6 lists them.

Tokenized-corpus SHA (first 12 hex): `fab530e59dea`; 169,741,563 training tokens per epoch in 41,440 chunks of 4,096.

Training configuration (all variants). `seq_len 4096 · batch 32 sequences (131,072 tokens/step) · 3,814 steps (499,908,608 tokens) · AdamW $\beta = (0.9, 0.95)$, wd 0.1 (dim ≥ 2 params only), fused · grad clip 1.0 · LR 6×10^{-4} , warmup 250 steps, cosine $\rightarrow 6 \times 10^{-5}$ · seed 42 · checkpoints every 100M tokens (five per run, all retained).`

The evaluation-weighting pitfall. Our training-time validation logger split the 488 evaluation windows into batches of `micro_batch_size` and returned the mean of per-batch means. For the four runs with micro-batch 16, $488 = 30 \times 16 + 8$, so the trailing 8 windows were weighted double per window; for the micro-batch-8 run, $488 = 61 \times 8$ exactly, so weighting was uniform. The dev slice is highly non-uniform — the trailing 8 windows average 1.60 nats against 3.15 for the rest — so the `mb=16` runs’ logged losses were biased low by ≈ 0.0247 while the `mb=8` run’s were unbiased. Reconstructing the logger’s arithmetic from per-window losses reproduces every logged value to 4 decimal places, confirming the mechanism. The bias is invisible when comparing runs that share a micro-batch setting and initially inverted the 3:1 ablation’s sign in our logs. All numbers in this

paper are recomputed from checkpoints under uniform per-token weighting; the logger is fixed in the released code. The lesson generalizes: *validation reductions must not depend on memory-layout settings*.

BLiMP

protocol.

`src/common/blimp_eval.py` reproduces the official BabyLM 2026 causal scorer directly against native checkpoints: summed log-probability over every non-BOS sentence token, temperature 1, macro-averaged by UID. Evaluator commit `3d57ddc8` and data revision `8d52da94` are pinned in the result artifact. Seven of 59,875 BLiMP pairs tied under every model and were resolved with a fixed seed; no supplement pairs tied. Full per-paradigm outputs are in `paper/figures/blimp_eval.json`.

Extra logging for the recursive models. Per-super-block gradient norms and per-recursion-pass activation RMS were logged throughout; activation RMS grows smoothly and roughly linearly across the three passes (end of training, Recursive 2:1: $1.06 \rightarrow 1.60 \rightarrow 2.12$ in super-block 0 and $2.79 \rightarrow 3.82 \rightarrow 5.01$ in super-block 1; Recursive 3:1: $1.29 \rightarrow 1.92 \rightarrow 2.51$ and $3.40 \rightarrow 4.79 \rightarrow 6.47$) with no sign of divergence, consistent with the $1/\sqrt{2D_{\text{eff}}}$ residual initialization.

Commands. `src/common/param_count.py` reproduces the parameter table; `src/common/smoke_test.py` is the pre-training gate; `modal_train.py::main --variant <name>` reproduces any single run; `modal_train.py::ckpt_eval` reproduces Tables 2, 3, and 5; `modal_train.py::blimp_eval_all` reproduces Table 4; `paper/figures/make_figures.py` regenerates Figures 1–2.

Model	Run ID	Wall clock	Throughput (tok/s)	Peak mem (GiB)	Micro-batch	PyTorch
GQA baseline	3rjjyyru	27.6 min	302,861	42.7	16	2.12.1+cu130
GDN 2:1	0k35h8so	18.8 min	443,084	33.6	16	2.13.0+cu130
GDN 3:1	obd8gifu	23.6 min	353,301	40.2	16	2.13.0+cu130
Recursive 2:1	vx5five3	46.2 min	180,660	70.0	16	2.13.0+cu130
Recursive 3:1	yiievwrz	62.1 min	134,334	45.4	8	2.13.0+cu130

Table 6: Per-run provenance. All runs: $1\times$ NVIDIA H100 80GB HBM3 (Modal), CUDA 13.0, Python 3.11, flash-linear-attention 0.5.1, FlashAttention-3 for GQA layers, bf16 autocast, fp32 master weights, fused AdamW.